

LAPORAN PROJECT SIMULASI

**PENGEMBANGAN SIMULATOR INTERAKTIF REPLIKASI DATABASE (DB-
REPLICASIM)**



DISUSUN OLEH :

Nama : Fikran

Nim : 105841119623

Kelas : RPL-6B

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

A. LATAR BELAKANG

Dalam era modern pengembangan aplikasi berskala besar (*scalable systems*), ketergantungan terhadap satu server database tunggal (*single point of failure*) menjadi risiko yang sangat besar. Jika server tersebut mengalami gangguan, seluruh sistem akan lumpuh. Oleh karena itu, arsitektur sistem terdistribusi menerapkan metode **Database Replication** (Replikasi Basis Data) untuk menyalin dan mendistribusikan data dari satu node ke node lainnya.

Meskipun replikasi menawarkan keuntungan berupa ketersediaan tinggi (*high availability*) dan toleransi kesalahan (*fault tolerance*), penerapannya memicu tantangan besar terkait konsistensi data, latensi transaksi, dan potensi konflik, sebagaimana dijelaskan dalam *CAP Theorem*.

Untuk memberikan pemahaman mendalam secara visual tanpa memerlukan overhead instalasi kluster database yang rumit, Saya mengembangkan **DB-ReplicaSim**, sebuah aplikasi simulator berbasis web interaktif. Simulator ini dirancang untuk mendemonstrasikan perilaku data, waktu respons, dan penanganan eror pada tiga arsitektur replikasi utama: Master-Slave Sinkron, Master-Slave Asinkron, dan Master-Master Asinkron.

B. TUJUAN PROJECT

1. Memvisualisasikan perbedaan mendasar antara mekanisme replikasi sinkron (*synchronous*) dan asinkron (*asynchronous*).
2. Menganalisis dampak dari *replication lag* (keterlambatan replikasi) terhadap konsistensi data yang dibaca oleh aplikasi klien.
3. Mensimulasikan skenario kegagalan sistem (*node crash*) untuk melihat ketahanan masing-masing topologi database.
4. Menyediakan alat peraga edukatif (*interactive learning tool*) yang ringan, portabel, dan mudah dipresentasikan.

C. SPESIFIKASI DAN ARSITEKTUR SIMULATOR

Simulator **DB-ReplicaSim** dibangun menggunakan teknologi *front-end* modern berbasis klien (*client-side*), yang memastikan aplikasi dapat berjalan secara instan di browser mana pun tanpa ketergantungan server lokal (*zero-dependency*).

1. Teknologi Utama:

- **HTML5 & JavaScript (ES6+):** Mengelola logika *state machine* database, simulasi *delay* jaringan via asynchronous timers (setTimeout), serta manipulasi DOM secara dinamis.
- **Tailwind CSS:** Digunakan untuk merancang antarmuka pengguna (UI) bertema *dark mode* yang bersih, responsif, dan profesional.
- **SVG (Scalable Vector Graphics) Dinamis:** Menghubungkan node database menggunakan garis vektor interaktif yang berubah warna dan animasi ketika proses replikasi sedang berlangsung.
- **FontAwesome Icons:** Menyediakan representasi visual ikonik untuk server, laptop klien, dan status jaringan.

2. Komponen Utama Antarmuka:

- *Control Panel (Sidebar):* Memilih mode replikasi, memasukkan query data tulis (key & value), mengatur durasi lag, serta tombol *Hardware Switch* untuk mematikan/menghidupkan server secara paksa.
- *Topology Visualizer (Main Workspace):* Pemetaan grafis interaktif yang menunjukkan posisi Client Apps, Master Server, dan Slave Server beserta nilai data riil di dalamnya.
- *Live Logs Terminal:* Konsol teks di bagian bawah yang mencatat setiap urutan proses internal beserta *timestamp* (waktu kejadian) untuk analisis kronologis transaksi.

D. ANALISIS DAN CARA KERJA MODE REPLIKASI

Berdasarkan pengujian pada simulator, berikut adalah analisis mendalam terhadap ketiga mode yang disimulasikan:

1. Master-Slave (Synchronous / Sinkron)

- **Mekanisme Kerja:** Saat klien mengirim perintah tulis ke Master, Master tidak langsung memberikan respons sukses kepada klien. Master mengirimkan data tersebut ke Slave terlebih dahulu dan *menahan* transaksi. Setelah Slave sukses menulis data dan mengirimkan sinyal konfirmasi (*Acknowledge / ACK*) kembali ke Master, barulah Master melakukan *commit* lokal dan menyatakan transaksi sukses kepada klien.

- Analisis Karakteristik:

Parameter	Karakteristik Hasil Simulasi
Konsistensi Data	<i>Strong Consistency</i> (Sangat Tinggi). Data di Master dan Slave selalu identik di setiap waktu.
Latensi Transaksi	Tinggi. Waktu respons klien sebanding dengan <i>Waktu Proses Master + Waktu Jaringan ke Slave + Waktu Proses Slave</i> .
Penanganan Kegagalan	Jika Slave dimatikan (<i>Crashed</i>) dalam simulator, Master akan menolak transaksi baru karena jaminan sinkronisasi tidak terpenuhi.

2. Master-Slave (Asynchronous / Asinkron)

- **Mekanisme Kerja:** Klien menulis data ke Master. Master langsung menyimpan data secara lokal dan seketika itu juga mengembalikan respons "Sukses" ke klien (~0.01 detik). Di latar belakang (*background process*), Master menjadwalkan pengiriman data ke Slave.
- Analisis Karakteristik:

Parameter	Karakteristik Hasil Simulasi
Konsistensi Data	<i>Eventual Consistency</i> . Ada jendela waktu (<i>replication lag</i>) di mana data di Slave masih berupa data lama (usang) sebelum sinkronisasi latar belakang selesai.
Latensi Transaksi	Sangat Rendah (Performa Optimal). Klien tidak perlu menunggu proses penulisan di server Slave.
Penanganan Kegagalan	Jika Master berhasil menulis tetapi mengalami <i>crash</i> tepat sebelum data dikirim ke Slave, maka data baru tersebut terancam hilang (<i>data loss</i>).

3. Master-Master (Asynchronous)

- **Mekanisme Kerja:** Klien menulis data ke Master. Master langsung menyimpan data secara lokal dan seketika itu juga mengembalikan respons "Sukses" ke klien (~0.01 detik). Di latar belakang (*background process*), Master menjadwalkan pengiriman data ke Slave.
- Analisis Karakteristik:

Parameter	Karakteristik Hasil Simulasi
-----------	------------------------------

Ketersediaan (<i>Availability</i>)	Sangat Tinggi. Jika Master 1 tumbang, Klien dapat mengalihkan target operasi tulis ke Master 2 secara penuh.
Risiko Konflik	Tinggi. Jika dua klien berbeda memperbarui kunci (<i>key</i>) yang sama pada Master 1 dan Master 2 di saat yang bersamaan, konflik data akan terjadi. Simulator memitigasi ini dengan pendekatan resolusi konflik sederhana berbasis urutan replikasi terakhir.

E. HASIL PENGUJIAN SKENARIO SIMULASI

Berikut adalah potongan kode logika utama dari simulator **DB-ReplicaSim**.

1. Sourcode

```
function executeWrite() {
    const targetNode = document.getElementById('write-
target').value;
    const key = document.getElementById('write-
key').value;
    const val = parseInt(document.getElementById('write-
value').value);

    addLog(`[Client] Mengirim request tulis: { ${key}:
${val} } ke server target ${targetNode.toUpperCase()}...`,
'client');

    // 1. Check if the target node is online
    if (!dbState[targetNode].online) {
        addLog(`[Client Error] Transaksi gagal! Target
Server ${targetNode.toUpperCase()} sedang offline / mati.`,
'error');
        return;
    }

    // Block visual animations during replication
    const activePath = targetNode === 'master1' ?
pathM1S1 : null;

    if (currentMode === 'ms-sync') {
        // SYNCHRONOUS FLOW
        addLog(`[${targetNode.toUpperCase()}] Memulai
transaksi sinkronous. Menunggu respons Slave...`, 'info');

        // Show replicating animation link
        pathM1S1.setAttribute('stroke', '#f59e0b');
        pathM1S1.classList.add('replicating-path');
```

```

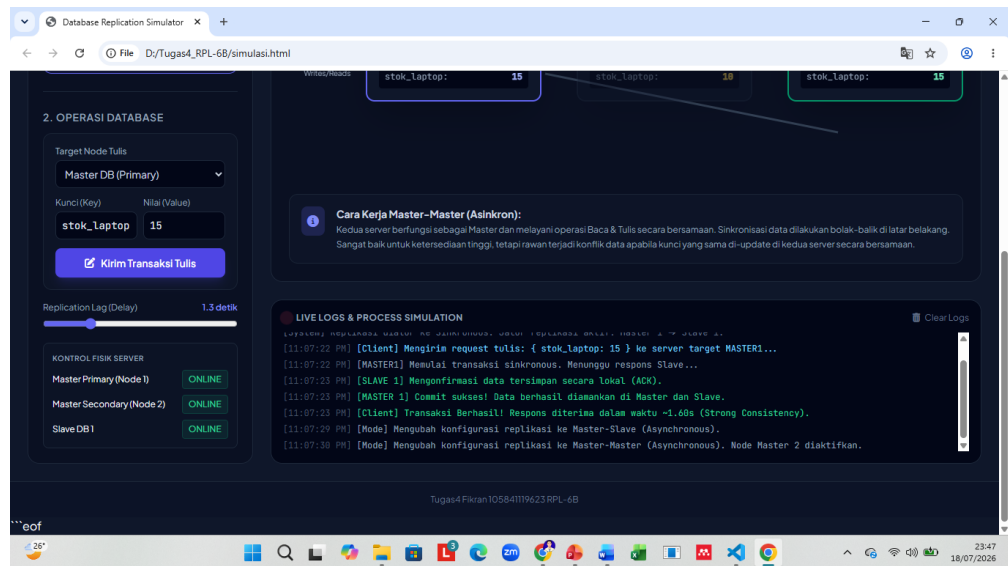
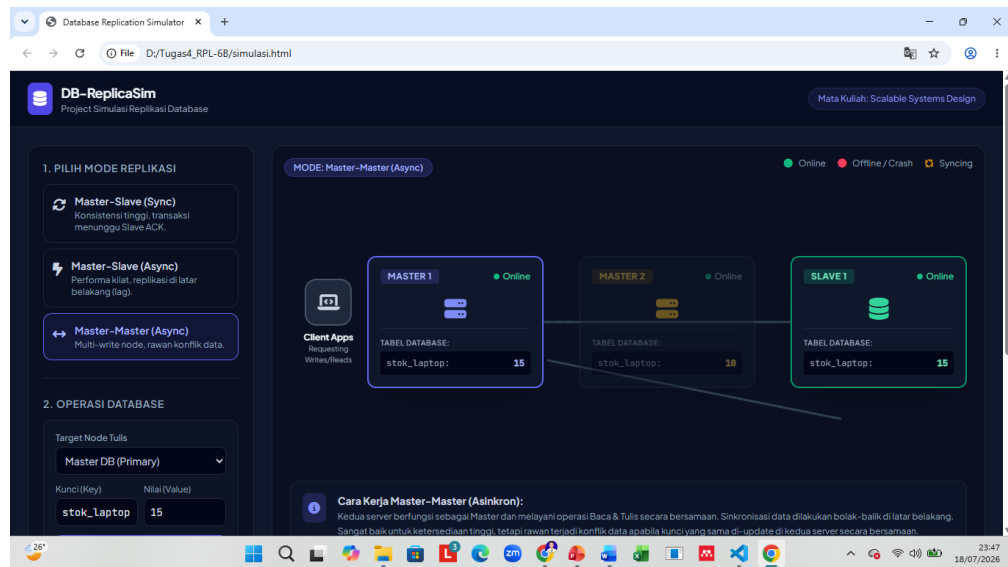
        setTimeout(() => {
            if (dbState.slave1.online) {
                // Slave active -> Saves data -> ACK to
master
                dbState.slave1.data[key] = val;
                document.getElementById('data-
s1').innerText = val;
                addLog(`[SLAVE 1] Mengonfirmasi data
tersimpan secara lokal (ACK).`, 'success');

                // Master commits locally
                dbState.master1.data[key] = val;
                document.getElementById('data-
m1').innerText = val;
                addLog(`[MASTER 1] Commit sukses! Data
berhasil diamankan di Master dan Slave.`, 'success');
                addLog(`[Client] Transaksi Berhasil!
Respons diterima dalam waktu ~${(lagValue + 0.1).toFixed(2)}s
(Strong Consistency).`, 'success');
            } else {
                // Slave is dead during a Sync
replication
                addLog(`[MASTER 1] Gagal melakukan
replikasi! SLAVE 1 tidak merespons (Offline).`, 'error');
                addLog(`[Client Error] Transaksi ditolak
oleh Master karena jaminan Sync gagal terpenuhi.`, 'error');
            }

            // Reset paths
            pathM1S1.setAttribute('stroke', '#334155');
            pathM1S1.classList.remove('replicating-
path');
        }, lagValue * 1000);

```

2. Output



F. KESIMPULAN

Melalui project simulator **DB-ReplicaSim**, Saya berhasil menyimpulkan bahwa tidak ada arsitektur replikasi tunggal yang sempurna untuk semua kasus. Pemilihan arsitektur harus disesuaikan dengan kebutuhan bisnis aplikasi:

1. **Master-Slave Sync** sangat cocok untuk sistem finansial atau perbankan yang mutlak membutuhkan akurasi data tinggi (*Strong Consistency*), meskipun harus mengorbankan kecepatan transaksi.
2. **Master-Slave Async** ideal untuk aplikasi sosial media, konten e-commerce, atau sistem blogging di mana kecepatan respons aplikasi jauh lebih diutamakan daripada sedikit keterlambatan pembaruan data di sisi pengguna lain.

3. **Master-Master Async** direkomendasikan untuk sistem global berskala besar yang membutuhkan redundansi tinggi di berbagai wilayah geografis guna menjamin sistem tidak pernah *down*.